

LAPORAN DOKUMENTASI

Aplikasi CRUD Mahasiswa

Nama: REVA ZAIDAN AZMI

Nim: 231011402702

Kelas: 05TPLE004

Daftar Isi

1. Overview Aplikasi
2. Struktur File
3. Dokumentasi HTML
4. Dokumentasi CSS
5. Dokumentasi JavaScript
6. Alur Kerja Aplikasi
7. Fitur-Fitur Utama
8. Penjelasan Code Penting

Overview Aplikasi

Deskripsi

Aplikasi **Sistem Data Mahasiswa** adalah web application untuk mengelola data mahasiswa dengan operasi CRUD:

- **Create:** Menambah data mahasiswa baru
- **Read:** Menampilkan daftar mahasiswa di tabel
- **Update:** Mengubah data mahasiswa yang sudah ada
- **Delete:** Menghapus data mahasiswa

Struktur File

```
tugas.html
├── HEAD (meta, styles)
├── BODY
│   └── container
│       ├── header (judul aplikasi)
│       ├── kartu form (form input)
│       └── kartu tabel (daftar data)
└── SCRIPT (JavaScript logic)
```

Dokumentasi HTML

1. Meta Information (Baris 2-5)

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
<title>Aplikasi CRUD Mahasiswa</title>
```

- Encoding UTF-8 untuk mendukung karakter Indonesia
- Viewport untuk responsive design
- Title untuk browser tab

2. Form Input (Baris 175-209)

```
<form id="mahasiswaForm">
  <div class="kotak-form">
    <div class="grup-input">
      <label for="nim">NIM</label>
      <input type="text" id="nim" required>
    </div>
    <!-- Input lainnya -->
  </div>
  <button type="submit" id="btnSubmit">Simpan Data</button>
</form>
```

Penjelasan:

- Form dengan ID mahasiswaForm untuk JavaScript hook
- 4 input field: NIM, Nama, Jurusan, Fakultas

- Attribute `required` untuk validasi HTML5
- Input Fakultas pakai `readonly` (hanya bisa di-auto-fill)
- Button submit untuk trigger penyimpanan

Field Input:

Field	Type	Validasi	Keterangan
NIM	text	required, unique	Nomor identitas mahasiswa
Nama	text	required	Nama lengkap
Jurusan	select	required	Pilihan dari 6 jurusan
Fakultas	text	readonly	Auto-fill dari mapping

3. Tabel Data (Baris 216-230)

```

<table>
  <thead>
    <tr>
      <th>No</th>
      <th>NIM</th>
      <th>Nama</th>
      <th>Jurusan</th>
      <th>Fakultas</th>
      <th>Aksi</th>
    </tr>
  </thead>
  <tbody id="listData">
    <!-- Data isi via JavaScript -->
  </tbody>
</table>

```

Penjelasan:

- `<tbody id="listData">` adalah target untuk rendering data via JavaScript
- Kolom Aksi berisi tombol Edit dan Hapus

Dokumentasi CSS

1. Reset & Base Styling (Baris 9-20)

```
* {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
  font-family: 'Segoe UI', sans-serif;  
}
```

- Memastikan consistency padding/margin di semua element
- Font yang clean dan modern

2. Form Styling (Baris 52-87)

```
.kotak-form {  
  display: grid;  
  grid-template-columns: repeat(2, 1fr);  
  gap: 15px;  
}  
  
.grup-input input:focus {  
  border-color: #3498db;  
}
```

- Grid layout 2 kolom untuk form fields
- Focus state dengan border biru untuk UX

3. Button Styling (Baris 94-114)

```
.tombol-biru { background-color: #2980b9; }  
.tombol-kuning { background-color: #f39c12; }  
.tombol-merah { background-color: #e74c3c; }  
.tombol-abu { background-color: #95a5a6; }
```

- 4 varian warna button untuk aksi berbeda
- Hover effect untuk interaktivitas

4. Table Styling (Baris 117-151)

```
table {  
  width: 100%;  
  border-collapse: collapse;  
}  
  
tr:hover {  
  background-color: #f8f9fa;  
}
```

- Full width table dengan responsive scroll
- Hover highlight untuk baris

5. Responsive Design (Baris 154-164)

```
@media (max-width: 600px) {  
  .kotak-form {  
    grid-template-columns: 1fr;  
  }  
  .baris-tombol {  
    flex-direction: column;  
  }  
  .btn {  
    width: 100%;  
  }  
}
```

- Layout berubah jadi 1 kolom di mobile
- Button full width
- Buttons stack vertical

Dokumentasi JavaScript

1. Data Mapping - Jurusan ke Fakultas (Baris 237-244)

```
const mapFakultas = {  
  "Teknik Informatika": "Fakultas Ilmu Komputer",  
  "Sistem Informasi": "Fakultas Ilmu Komputer",  
  "Teknik Elektro": "Fakultas Teknik",  
  "Teknik Industri": "Fakultas Teknik",  
  "Manajemen": "Fakultas Ekonomi",  
  "Akuntansi": "Fakultas Ekonomi"  
};
```

Fungsi: Mapping otomatis jurusan ke fakultas

Kegunaan: Auto-fill field Fakultas tanpa user input manual

2. Auto-fill Fakultas (Baris 247-256)

```
document.getElementById('jurusan').addEventListener('change',
function() {
    const jurusanPilihan = this.value;
    const fakultasInput = document.getElementById('fakultas');

    if (mapFakultas[jurusanPilihan]) {
        fakultasInput.value = mapFakultas[jurusanPilihan];
    } else {
        fakultasInput.value = '';
    }
});
```

Event: change pada select Jurusan

Logic:

1. Ambil nilai jurusan yang dipilih
2. Cari di object mapFakultas
3. Jika ada, isi field Fakultas
4. Jika tidak ada, kosongkan field

3. State Management (Baris 259-261)

```
let mahasiswaList = JSON.parse(localStorage.getItem('mhs_data')) ||
[];
let isEditing = false;
let indexEdit = null;
```

Variabel Global:

Variabel	Tipe	Fungsi
mahasiswaList	Array	Array berisi semua data mahasiswa
isEditing	Boolean	Flag untuk mode edit

Variabel	Tipe	Fungsi
indexEdit	Number/null	Index array saat edit

LocalStorage Usage: Data dimuat dari localStorage, jika tidak ada gunakan array kosong

4. Render Tabel Function (Baris 270-293)

```
function renderTable() {
    listContainer.innerHTML = '';

    if (mahasiswaList.length === 0) {
        listContainer.innerHTML = `<tr><td colspan="6">Belum ada data</td></tr>`;
        return;
    }

    mahasiswaList.forEach((mhs, idx) => {
        const tr = document.createElement('tr');
        tr.innerHTML = `
            <td>${idx + 1}</td>
            <td>${mhs.nim}</td>
            <td>${mhs.nama}</td>
            <td>${mhs.jurusan}</td>
            <td>${mhs.fakultas}</td>
            <td class="tombol-aksi">
                <button onclick="setupEdit (${idx}) ">Edit</button>
                <button onclick="hapusData (${idx}) ">Hapus</button>
            </td>
        `;
        listContainer.appendChild(tr);
    });
}
```

Fungsi: Menampilkan data mahasiswa di tabel

Alur:

1. Kosongkan container tabel
2. Cek apakah ada data
3. Jika tidak ada, tampilkan pesan kosong
4. Jika ada, loop setiap mahasiswa
5. Buat row baru dan append ke tabel
6. Tambahkan tombol Edit & Hapus untuk setiap row

5. Form Submit Handler (Baris 296-326)

```
form.addEventListener('submit', function(e) {
    e.preventDefault();

    const nim = document.getElementById('nim').value.trim();
    const nama = document.getElementById('nama').value.trim();
    const jurusan = document.getElementById('jurusan').value;
    const fakultas = document.getElementById('fakultas').value;

    // Validasi duplikat NIM
    const nimExist = mahasiswaList.find((item, index) =>
        item.nim === nim && index !== indexEdit
    );
    if (nimExist) {
        alert('Waduh, NIM ini udah dipake!');
        return;
    }

    if (isEditing) {
        mahasiswaList[indexEdit] = { nim, nama, jurusan, fakultas };
    } else {
        mahasiswaList.push({ nim, nama, jurusan, fakultas });
    }

    localStorage.setItem('mhs_data',
        JSON.stringify(mahasiswaList));
    renderTable();
    form.reset();
});
```

Event: `submit` pada form

Alur:

1. Prevent default form submission
2. Ambil nilai dari semua input field
3. **Validasi Duplikat NIM:**
 - Cari NIM yang sama di array
 - Exclude index yang sedang diedit
 - Jika ada duplikat, tampilkan alert & return
4. **Mode Edit atau Add:**
 - Jika edit: update data di array
 - Jika add: push data baru
5. Simpan ke `localStorage`
6. Re-render tabel
7. Reset form

6. Setup Edit Function (Baris 329-344)

```
function setupEdit(index) {
    isEditing = true;
    indexEdit = index;

    const mhs = mahasiswaList[index];
    document.getElementById('nim').value = mhs.nim;
    document.getElementById('nama').value = mhs.nama;
    document.getElementById('jurusan').value = mhs.jurusan;
    document.getElementById('fakultas').value = mhs.fakultas;

    formTitle.innerText = "Edit Data Mahasiswa";
    btnSubmit.innerText = "Simpan Perubahan";
    btnSubmit.className = "btn tombol-kuning";
    btnBatal.style.display = "inline-block";
}
```

Fungsi: Setup form untuk mode edit

Alur:

1. Set flag `isEditing = true`
2. Catat index data yang akan diedit
3. Ambil data dari array
4. Isi form dengan data tersebut
5. Ubah UI:
 - Title: "Edit Data Mahasiswa"
 - Button: "Simpan Perubahan" (warna kuning)
 - Tampilkan tombol "Batal Edit"

7. Batalkan Edit Function (Baris 347-356)

```
function batalEdit() {
    isEditing = false;
    indexEdit = null;
    form.reset();

    formTitle.innerText = "Tambah Data Mahasiswa";
    btnSubmit.innerText = "Simpan Data";
    btnSubmit.className = "btn tombol-biru";
    btnBatal.style.display = "none";
}
```

Fungsi: Reset form ke mode tambah

Alur:

1. Set `isEditing = false`
2. Clear `indexEdit`
3. Reset semua input field
4. Kembalikan UI ke state awal

8. Hapus Data Function (Baris 364-378)

```
function hapusData(index) {  
    const yakin = confirm(`Yakin mau menghapus  
${mahasiswaList[index].nama}?`);  
    if (yakin) {  
        mahasiswaList.splice(index, 1);  
        localStorage.setItem('mhs_data',  
JSON.stringify(mahasiswaList));  
        renderTable();  
  
        if (isEditing && indexEdit === index) {  
            batalEdit();  
        }  
  
        alert('Data berhasil didelete.');
```

Fungsi: Menghapus data mahasiswa

Alur:

1. Tampilkan konfirmasi dialog
2. Jika user confirm:
 - Hapus dari array (splice)
 - Update localStorage
 - Re-render tabel
 - Jika sedang edit data itu, batalkan edit
 - Tampilkan alert success

Alur Kerja Aplikasi

Startup (Halaman Dibuka)

1. Load data dari localStorage
2. Set state: `isEditing = false, indexEdit = null`
3. Render tabel dengan data yang dimuat

Tambah Data

User mengisi form -> Klik "Simpan Data" ->
Validasi duplikat NIM -> Push ke array ->
Simpan ke localStorage -> Re-render tabel ->
Reset form

Edit Data

User klik "Edit" di tabel -> `setupEdit(index)` ->
Form diisi data lama -> User ubah data ->
Klik "Simpan Perubahan" -> Update array ->
Simpan ke localStorage -> Re-render tabel ->
`batalEdit()`

Hapus Data

User klik "Hapus" -> Konfirmasi ->
`Splice` dari array -> Simpan localStorage ->
Re-render tabel

Auto-fill Fakultas

User pilih Jurusan -> Event 'change' trigger ->
Lookup di `mapFakultas` -> Auto-fill Fakultas

Fitur-Fitur Utama

1. CRUD Operations

- **Create:** Tombol "Simpan Data" menambah data baru
- **Read:** Tabel menampilkan semua data
- **Update:** Tombol "Edit" mengubah data existing
- **Delete:** Tombol "Hapus" menghapus data dengan konfirmasi

2. Validasi

- Field required (HTML5)
- Duplikat NIM tidak diperbolehkan
- Trim whitespace pada input

3. User Experience

- Form title berubah saat edit
- Button color berbeda (biru=add, kuning=edit, merah=delete)
- Tombol "Batal Edit" hanya tampil saat edit
- Hover effect pada tabel & button
- Confirmation dialog saat hapus

4. Data Persistence

- LocalStorage menyimpan data
- Data tidak hilang saat refresh/tutup browser
- Data tersimpan di device local

5. Automation

- Auto-fill Fakultas berdasarkan Jurusan
- Mapping 6 jurusan ke 3 fakultas

Penjelasan Code Penting

Code 1: Validasi Duplikat NIM

File: tugas.html, Baris 305-309

```
const nimExist = mahasiswaList.find((item, index) =>
  item.nim === nim && index !== indexEdit
);
if (nimExist) {
  alert('Waduh, NIM ini udah dipake sama mahasiswa lain!');
  return;
}
```

Penjelasan:

- `.find()` mencari elemen pertama yang match kondisi
- Cek `item.nim === nim`: NIM ada di database
- Cek `index !== indexEdit`: Tapi bukan data yang sedang diedit
- Jika duplikat, alert & return tanpa simpan
- **Benefit**: Mencegah duplicate key di database

Code 2: Toggle Edit Mode

File: tugas.html, Baris 311-315

```
if (isEditing) {
  mahasiswaList[indexEdit] = { nim, nama, jurusan, fakultas };
  alert('Data mahasiswa berhasil di-update!');
  batalEdit();
} else {
  mahasiswaList.push({ nim, nama, jurusan, fakultas });
  alert('Data mahasiswa berhasil disimpan!');
}
```

Penjelasan:

- Flag `isEditing` mengontrol behavior submit button
- Mode **Edit**: Update existing object di array

- Mode **Add**: Push object baru ke array
- Setelah edit, otomatis `batalEdit()` untuk reset UI

Code 3: Auto-fill dengan Event Listener

File: tugas.html, Baris 247-256

```
document.getElementById('jurusan').addEventListener('change',
function() {
    const jurusanPilihan = this.value;
    const fakultasInput = document.getElementById('fakultas');

    if (mapFakultas[jurusanPilihan]) {
        fakultasInput.value = mapFakultas[jurusanPilihan];
    } else {
        fakultasInput.value = '';
    }
});
```

Penjelasan:

- Event listener menunggu `change` event pada select
- Ambil value dari dropdown
- Lookup di object mapping
- Set value input Fakultas
- **Benefit:** User-friendly, mengurangi manual input

Code 4: Render Tabel Dinamis

File: tugas.html, Baris 278-292

```
mahasiswaList.forEach((mhs, idx) => {
    const tr = document.createElement('tr');
    tr.innerHTML = `
        <td>${idx + 1}</td>
        <td>${mhs.nim}</td>
        <td>${mhs.nama}</td>
        <td>${mhs.jurusan}</td>
        <td>${mhs.fakultas}</td>
        <td class="tombol-aksi">
```

```

                <button onclick="setupEdit(${idx})">Edit</button>
                <button onclick="hapusData(${idx})">Hapus</button>
            </td>
        `;
        listContainer.appendChild(tr);
    });

```

Penjelasan:

- `.forEach()` loop setiap data mahasiswa
- `.createElement('tr')` buat row baru
- Template string untuk HTML dinamis
- `${idx + 1}` nomor urut (1-based)
- Inline onclick untuk Edit & Hapus
- `.appendChild()` masukkan row ke tabel
- **Benefit:** Render otomatis sesuai data di array

Code 5: LocalStorage Persistence

File: tugas.html, Baris 259 & 323

```

// Baris 259 - Load saat startup
let mahasiswaList = JSON.parse(localStorage.getItem('mhs_data')) ||
[];

// Baris 323 - Save setiap kali ada perubahan
localStorage.setItem('mhs_data', JSON.stringify(mahasiswaList));

```

Penjelasan:

- **Load:** `localStorage.getItem()` ambil data, `JSON.parse()` konversi string ke object
- **Save:** `JSON.stringify()` konversi object ke string, `setItem()` simpan
- `|| []` fallback ke array kosong jika belum ada data
- **Benefit:** Data persistent, tidak hilang saat refresh

Potential Bug & Enhancement

Bug yang Mungkin Terjadi:

1. Jika edit data kemudian hapus data lain dengan index lebih kecil, indexEdit bisa salah
 - **Fix:** Perlu update indexEdit setelah splice
2. Input NIM bisa accept special character
 - **Fix:** Tambah regex validation untuk hanya angka

Enhancement yang Bisa Ditambah:

1. Search/Filter data mahasiswa
2. Sort tabel (berdasarkan NIM, Nama, Jurusan)
3. Export data ke Excel/PDF
4. Backup & Import data
5. Validation format NIM (harus 12 digit)
6. Pagination jika data banyak
7. Konfirmasi dialog lebih elegant (modal popup)

Statistik Code

Aspek	Detail
Total Baris	384
HTML Baris	166
CSS Baris	165
JavaScript Baris	~120
Jumlah Function	5 function
Global Variable	3 variable
Event Listener	3 listener
LocalStorage Key	1 key (mhs_data)

Teknologi yang Digunakan

Teknologi	Versi	Fungsi
HTML	5	Struktur dokumen
CSS	3	Styling & responsive
JavaScript	ES6+	Logic & interaktivitas
LocalStorage API	Web API	Penyimpanan data
DOM API	Web API	Manipulasi DOM

Testing Checklist

- Tambah data mahasiswa baru
- Cek data muncul di tabel
- Edit data existing
- Cek perubahan ter-reflect di tabel
- Hapus data (confirm dialog)
- Refresh halaman, cek data masih ada
- Coba tambah NIM duplikat (harus error)
- Ubah jurusan, cek fakultas auto-fill
- Test di mobile/resize browser
- Clear localStorage, check aplikasi reset

Kesimpulan

Aplikasi CRUD Mahasiswa adalah contoh bagus dari:

- Single Page Application (SPA)
- Client-side data management
- Event-driven programming
- DOM manipulation
- LocalStorage usage
- Responsive web design
- Form validation & error handling

Link Repository

<https://github.com/zaidanity/Tugas-Pemrograman-Web1>